Sujet de Projet Final – Application E-Commerce (React + TypeScript)

Contexte général

Dans le cadre du module **React & TypeScript**, vous devez réaliser une application complète d'e-commerce en utilisant :

- React 18+
- TypeScript
- Vite
- React Router
- Context API (ou Zustand/Recoil si vous préférez, mais optionnel)
- Chakra UI
- ESLint + conventions vues en cours
- L'API fournie (Swagger)

Ce projet est **individuel**

Vous disposez de **3 mois** pour livrer une application fonctionnelle, testée, cohérente et bien présentée.

Rendus attendus pour le 3 mars 2026

L'API est disponible ici : https://github.com/KennochaJilian/18_06_ecom

Objectifs pédagogiques

Ce projet doit démontrer votre capacité à :

- Créer une architecture front moderne et scalable
- Implémenter une navigation complète (React Router)
- Manipuler une API REST (CRUD, auth, JWT, gestion des erreurs)
- Gérer un état global (user connecté, panier…)
- Concevoir une UI moderne et responsive avec Chakra UI
- Utiliser TypeScript proprement : types, interfaces, modèles, services
- Respecter une architecture claire en modules/composants/services

- Mettre en place une logique d'authentification sécurisée (JWT)
- Documenter une application web (README + résumé pédagogique)

Livrables obligatoires

1. **Application complète fonctionnelle**,
2. **Prototype / Maquette sur Figma ou Adobe XD**
    - Pages obligatoires : Home, Détail produit, Login, Profil, Panier
    - Design responsive obligatoire
3. **Résumé pédagogique (1 à 2 pages)**
   Vous expliquerez ce que vous avez appris sur :
    - CSR vs SSR
    - Les verbes HTTP
    - DNS et fonctionnement d'un nom de domaine
    - Architecture microservices vs monolithique
    - Semver, git, etc.
4. **Code source propre, structuré, typé, commenté**
5. **README complet**
    - Présentation du projet
    - Installation
    - Choix techniques
    - Architecture
    - Difficultés rencontrées

Fonctionnalités obligatoires (Minimum Viable Product)

1. Page d'accueil

→ Récupérer les produits via l'endpoint :

GET /api/products

Chaque produit doit apparaître sous forme de **card** (Chakra UI).
Chaque card doit afficher :

- Une image du produit
- Le nom
- Le prix
- Un bouton “Voir le détail”

Possibilité de rechercher (query ?search=).

2. Page de détail d’un produit

→ Récupérer via :

GET /api/products/{product_id}

Afficher au minimum :

- Une image (issue de Images[0].link)
- Le nom
- Le prix
- Description
- Stock
- Un bouton “Ajouter au panier”

3. Authentification (login/signup)

Inscription

Via POST /api/users/signup

Connexion

Via POST /api/users/signin

Le token JWT doit être stocké dans :

- localStorage (le plus simple), ou
- un cookie HTTPOnly si vous souhaitez aller plus loin

Une fois logué, l’utilisateur doit être redirigé vers la page d’accueil.

👩‍💼 **4. Page Profil**

→ Récupérer via :

GET /api/users/{user_id} (token obligatoire)

La page doit permettre :

- D'afficher les infos : nom, email
- De modifier les infos (PUT /users/{id})
- Bonus : gérer adresses / téléphone / moyens de paiement

5. Panier + passage de commande

Le panier est géré côté API :

- Récupération :
  GET /api/carts/user/{user_id}
- Ajouter un produit :
  POST /api/carts/{cart_id}/products/{product_id}
- Supprimer un produit :
  DELETE /api/carts/{cart_id}/products/{product_id}

Vous devez :

- Afficher la liste des produits du panier
- Afficher le total
- Permettre de supprimer un article
- Permettre de finaliser la commande (même si l'API ne propose pas un vrai paiement, un écran de confirmation suffit)

Fonctionnalités Bonus (pour aller chercher une meilleure note)

- Filtre par catégorie (/api/categories)
- Page Admin (si rôle admin)
- Gestion des adresses + paiement via endpoints disponibles
- Pagination / infinite scroll
- Animations Chakra UI
- Dark mode
- Gestion du stock dynamique
- Tests unitaires (Vitest)

- Déploiement (Vercel / Netlify)

Architecture recommandée

```
src/
 ├─ components/
 ├─ pages/
 ├─ hooks/
 ├─ context/
 ├─ services/ (appels API typés)
 ├─ types/ (interfaces TS)
 ├─ utils/
 ├─ router/
 └─ App.tsx
```

Services API par exemple :

```
services/
 ├─ api.ts
 ├─ auth.service.ts
 ├─ product.service.ts
 └─ cart.service.ts
```

Évaluation

Critère	**Coefficient**
Fonctionnalités obligatoires	40
Qualité du code (TS, structure, réutilisabilité)	20
UI/UX + maquette	5
Bonne utilisation API REST	10
Auth & routing	5
Panier & gestion état global	5
Résumé pédagogique	10
README	5